
Projektbericht

Full Stack Handwerk

Prediction Analytics Endpoint

Erweiterung des Kursblocks *Datenbanken & SQL*

Verfasserin: Alina Wagner
Matrikelnummer: 5083562
Dozent: Prof. Dr.-Ing. Mark Schutera
Datum: 30.06.2026

Dualer Partner: ZDF
DHBW Ravensburg – Studiengang Data Science & KI

Inhaltsverzeichnis

1 Ausgangslage und Motivation	2
2 Datenbankkonzepte und analytische Abfragen	2
2.1 Vom einfachen SELECT zur Aggregation	2
2.2 HAVING – Filterung auf Gruppenebene	2
2.3 Zeitreihen und die Rolle des Zeitstempels	3
2.4 Window Functions – Kontext ohne Informationsverlust	3
2.5 Filterparameter und ihre analytische Bedeutung	3
3 Performance-Analyse	4
3.1 Das Problem: Sequential Scan	4
3.2 Die Lösung: B-Tree-Index	4
3.3 Messung mit EXPLAIN ANALYZE	4
4 Integration in die Infrastruktur	4
4.1 Automatisierter Run per systemd-Timer	4
4.2 Messbarkeit: Baseline-Vergleich zwischen Modellversionen	5
5 Probleme, Ergebnis und Ausblick	5
5.1 Aufgetretene Probleme	5
5.2 Ergebnis	6
5.3 Ausblick	6
Quellen	6

1 Ausgangslage und Motivation

PixelWise klassifiziert handgeschriebene Ziffern mithilfe eines Machine-Learning-Modells und speichert seit dem Kursblock *Datenbanken & SQL* jede Klassifikation in einer PostgreSQL-Datenbank. Nutzer zeichnen dabei eine Ziffer, das Modell ordnet ihr eine Vorhersage samt Konfidenzwert zu, und dieses Ergebnis wird zusammen mit Zeitstempel und Modellversion persistiert. Die Grundlage bildet die Tabelle `predictions`, deren Struktur in Tabelle 1 dargestellt ist.

Tabelle 1: Datenbankschema der Tabelle `predictions`

Spalte	Datentyp	Bedeutung
<code>id</code>	Integer (PK)	Eindeutiger Primärschlüssel
<code>prediction</code>	String	Vorhergesagte Ziffer (1–9)
<code>confidence</code>	Float	Konfidenz des Modells (0.0–1.0)
<code>model_version</code>	String	Verwendete Modellversion
<code>created_at</code>	DateTime	Zeitstempel der Klassifikation

Der einzige lesende Endpoint im Auslieferungszustand, `GET /results`, gibt lediglich die letzten 20 Einträge chronologisch aus – ohne Aggregation, ohne analytischen Mehrwert. Das reicht aus, um zu prüfen ob die Datenbank korrekt beschrieben wird, beantwortet aber keine einzige inhaltliche Frage. Ist das Modell bei bestimmten Ziffern systematisch unsicherer? Hat sich das Verhalten nach einem Modellupdate verändert? Gibt es Zeiträume mit auffällig vielen Fehlerklassifikationen? Solche Fragen lassen sich aus den Rohdaten nicht beantworten, solange sie nicht ausgewertet werden.

Diese Erweiterung setzt genau hier an: Mit `GET /stats` wird die Datenbank von einem reinen Speicher zu einer Erkenntnisquelle, die aggregierte Auswertungen liefert – wie oft welche Ziffer erkannt wurde, wie sicher das Modell dabei war, und wie sich das über die Zeit entwickelt hat. Damit wird erstmals möglich, das Modellverhalten datengetrieben zu bewerten und Veränderungen nach einem Modellwechsel direkt zu messen.

Der vollständige Quellcode ist im GitHub-Repository verfügbar:

<https://github.com/alina.wagner1805-crypto/Full-Stack-Handwerk>

2 Datenbankkonzepte und analytische Abfragen

2.1 Vom einfachen SELECT zur Aggregation

Der Kurs behandelt `SELECT`-Abfragen, die einzelne Zeilen zurückgeben. Für eine Auswertung des Modellverhaltens reicht das nicht – es braucht Abfragen, die über viele Zeilen hinweg rechnen. Genau das leisten Aggregatfunktionen: `COUNT` zählt wie viele Predictions für eine bestimmte Ziffer existieren, `AVG` berechnet die durchschnittliche Konfidenz über alle Einträge einer Gruppe. In Kombination mit `GROUP BY` entsteht daraus eine Auswertung, die das Verhalten des Modells pro Ziffer sichtbar macht – also ob das Modell eine Drei konsistent sicher erkennt, oder ob die Konfidenz stark streut.

2.2 HAVING – Filterung auf Gruppenebene

Eine der zentralen Designentscheidungen betrifft die Filterung unsicherer Zifferngruppen. Intuitiv würde man dafür `WHERE` verwenden – also alle Predictions mit niedriger Konfidenz vor der Berechnung herausfiltern. Das führt jedoch zu einem Denkfehler: Wenn man Predictions mit niedriger Konfidenz vor der Aggregation entfernt, verändert

man die Datenbasis und verfälscht damit den berechneten Durchschnitt. Eine Ziffer, bei der das Modell oft unsicher ist, würde dann künstlich besser aussehen als sie ist, weil die schlechten Werte bereits herausgefiltert wurden.

Die korrekte Lösung ist `HAVING`: Diese Klausel filtert nicht einzelne Zeilen, sondern ganze Gruppen – und zwar *nach* der Aggregation. Das bedeutet, alle Predictions fließen vollständig in die Berechnung ein; erst danach werden Gruppen verworfen, deren Durchschnitt den Schwellenwert unterschreitet. Das Ergebnis ist repräsentativ.

2.3 Zeitreihen und die Rolle des Zeitstempels

Jede Prediction trägt einen Zeitstempel. Dieser wird genutzt, um das Modellverhalten über die Zeit aufzuschlüsseln – also nicht nur *wie gut* das Modell insgesamt ist, sondern *wann* sich etwas verändert hat. Dafür wird der genaue Zeitstempel auf den reinen Datumsteil reduziert, sodass alle Predictions eines Tages zusammengefasst werden können. Das Ergebnis ist eine Zeitreihe, die tagesweise zeigt wie viele Klassifikationen stattgefunden haben und wie hoch die Durchschnittskonfidenz war.

Diese Zeitreihe ist besonders wertvoll in Kombination mit dem Filterparameter für die Modellversion: Man kann damit gezielt den Zeitraum *nach* einem Modellwechsel betrachten und sehen, ob sich die Konfidenz verändert hat.

2.4 Window Functions – Kontext ohne Informationsverlust

Eine Limitation von `GROUP BY` ist, dass dabei alle Originalzeilen verloren gehen – man sieht nur noch das Aggregat. Window Functions lösen dieses Problem: Sie berechnen einen Wert relativ zu einer Gruppe, ohne die einzelnen Zeilen zu kollabieren. Im Kontext von PixelWise bedeutet das: Man kann jeder Prediction ihren Rang innerhalb der eigenen Zifferngruppe zuweisen – geordnet nach Konfidenz. Damit lässt sich zum Beispiel erkennen, ob die konfidenzstärksten Predictions einer Ziffer systematisch zu bestimmten Tageszeiten auftreten, oder ob Ausreißer nach oben und unten gleichmäßig verteilt sind.

2.5 Filterparameter und ihre analytische Bedeutung

Der Endpoint unterstützt drei optionale Filterparameter, die einzeln und kombiniert verwendet werden können. Tabelle 2 gibt einen Überblick.

Tabelle 2: Filterparameter des `GET /stats`-Endpoints

Parameter	Analytische Bedeutung
<code>model_version</code>	Schränkt die Auswertung auf eine bestimmte Modellversion ein – essenziell, um Modellvergleiche nicht durch vermischte Daten zu verfälschen.
<code>since</code>	Begrenzt die Auswertung auf Predictions ab einem bestimmten Datum – ermöglicht gezielte Analysen nach Ereignissen wie Modellupdates.
<code>confidence_below</code>	Gibt nur Zifferngruppen unter einem Konfidenz-Schwellenwert aus, mittels <code>HAVING</code> statt <code>WHERE</code> , um das Ergebnis repräsentativ zu halten.

3 Performance-Analyse

3.1 Das Problem: Sequential Scan

Ohne Index arbeitet PostgreSQL bei jeder Abfrage mit einem sogenannten *Sequential Scan*: Die Datenbank liest die gesamte `predictions`-Tabelle zeilenweise von Anfang bis Ende durch, um die relevanten Einträge zu finden. Das Entscheidende dabei ist das Laufzeitverhalten: Es wächst linear mit der Anzahl der Zeilen. Das bedeutet konkret – bei 1.000 Predictions dauert eine gefilterte Abfrage etwa 1 ms, bei 10.000 Predictions bereits 10 ms, bei 100.000 Predictions 100 ms und bei einer Million Predictions rund eine Sekunde. Für eine Anwendung wie PixelWise, die täglich neue Predictions speichert und deren Datenbestand kontinuierlich wächst, ist das ein strukturelles Problem: Die Abfragezeiten verschlechtern sich automatisch mit der Zeit, ohne dass sich an der Abfrage selbst etwas geändert hat.

3.2 Die Lösung: B-Tree-Index

Ein Datenbankindex auf der Spalte `created_at` behebt dieses Problem grundlegend. PostgreSQL legt dabei eine sortierte Baumstruktur – einen sogenannten B-Tree – an, der die Zeitstempel geordnet speichert. Bei einer gefilterten Abfrage wie `?since=2026-05-01` kann die Datenbank damit direkt zu den relevanten Einträgen springen, ohne die gesamte Tabelle zu lesen. Das Laufzeitverhalten ändert sich dadurch von linear zu logarithmisch: Bei 1.000 Predictions sind es etwa 10 Vergleiche im Index, bei 10.000 etwa 13, bei 100.000 etwa 17 und bei einer Million etwa 20. Die Abfragezeit wächst also kaum noch, egal wie viele Predictions gespeichert werden. Das macht den Endpoint langfristig skalierbar – ohne Änderungen am Code.

Ein Index verursacht allerdings bei jedem `INSERT` einen geringen Schreiboverhead, weil die Baumstruktur aktualisiert werden muss. Für PixelWise, das deutlich öfter liest als schreibt, ist dieser Kompromiss klar vertretbar.

3.3 Messung mit EXPLAIN ANALYZE

`EXPLAIN ANALYZE` ist ein PostgreSQL-Werkzeug, das eine Abfrage tatsächlich ausführt und dabei den gewählten Ausführungsplan protokolliert – inklusive der realen Laufzeit. Das ist wichtig, weil PostgreSQL intern verschiedene Strategien kennt und je nach Datenlage selbst entscheidet, welche es verwendet. Ohne Index zeigt `EXPLAIN ANALYZE` einen Sequential Scan mit linear wachsenden Kosten. Nach dem Anlegen des Index wechselt PostgreSQL automatisch auf einen Index Scan, was sich in den gemessenen Ausführungszeiten direkt widerspiegelt. Ausführliche Messergebnisse und vollständige Query Plans sind im Repository unter [docs/performance.md](#) dokumentiert.

4 Integration in die Infrastruktur

4.1 Automatisierter Run per systemd-Timer

Ein Endpoint der nur auf Anfrage antwortet, kann keine Veränderungen über Zeit erkennen. Um das Modellverhalten kontinuierlich beobachten zu können, wird `GET /stats` täglich automatisch aufgerufen – per systemd-Timer, der auf demselben Server läuft wie die FastAPI-Anwendung. Das Ergebnis wird als datierte JSON-Datei gespeichert, sodass zu jedem Tag ein Snapshot des Modellzustands vorliegt.

Der Aufbau folgt dem Muster aus dem Kursblock *The Backend: APIs & Services*: Eine Service-Unit führt das Shell-Skript `collect_stats.sh` als einmaligen Prozess aus, eine Timer-Unit löst diesen Service täglich um 02:00 Uhr aus. Die Option `Persistent=true`

stellt dabei sicher, dass ein verpasster Run – etwa nach einem Serverneustart – beim nächsten Start automatisch nachgeholt wird. Dadurch entstehen keine Lücken in der Snapshot-Historie, selbst wenn der Server gelegentlich neu gestartet wird. Der vollständige Aufbau ist unter [docs/monitoring.md](#) dokumentiert.

4.2 Messbarkeit: Baseline-Vergleich zwischen Modellversionen

Die gespeicherten Snapshots bilden die Grundlage für einen direkten Vorher-Nachher-Vergleich. Das Skript `compare_stats.py` liest zwei JSON-Dateien ein – einen Snapshot vor und einen nach einem Ereignis, etwa einem Modellwechsel – und gibt die Differenz der Durchschnittskonfidenz je Ziffer aus.

Abbildung 1 zeigt das tatsächliche Ergebnis dieses Vergleichs nach dem Einspielen von v2-Testdaten: Die Gesamtkonfidenz steigt von 0,8144 auf 0,8607, wobei die zuvor schwächsten Ziffern – 6, 8 und 9 – mit Abstand am stärksten profitieren.

Ziffer	Vorher	Nachher	Delta
1	0.8741	0.9044	+0.0303
2	0.8735	0.9018	+0.0283
3	0.8730	0.8966	+0.0236
4	0.8773	0.8946	+0.0173
5	0.8793	0.8887	+0.0094
6	0.6838	0.7789	+0.0951
7	0.8744	0.9082	+0.0338
8	0.6715	0.7686	+0.0971
9	0.6585	0.7732	+0.1147

Abbildung 1: Tatsächliches Ergebnis von `compare_stats.py` nach Einspielen von v2-Testdaten

Dies ist ein entscheidender Unterschied zum Ausgangszustand: Bisher konnte man nur beobachten, was das Modell *momentan* tut. Mit dem Snapshot-System lässt sich erstmals nachvollziehen, wie sich das Modellverhalten *verändert* hat – messbar, reproduzierbar und ohne manuellen Aufwand. Der Ablauf vom Einspielen der Testdaten bis zum Vergleich ist mit `demo/live_demo.sh` in einem Schritt reproduzierbar.

Zum eigenständigen Ausprobieren steht nach dem Start des Servers (siehe README im Repository) die interaktive Swagger-Oberfläche unter <http://localhost:8000/docs> im Browser zur Verfügung. Dort

lassen sich alle Endpunkte – inklusive `GET /stats` und seiner Filterparameter – direkt über die Schaltfläche *Try it out* ausführen und die jeweilige Response sofort einsehen, ohne ein separates Tool wie `curl` zu benötigen.

Das ist ein entscheidender Unterschied zum Ausgangszustand: Bisher konnte man nur beobachten, was das Modell *gerade* tut. Mit dem Snapshot-System lässt sich erstmals nachvollziehen, wie sich das Modellverhalten *verändert* hat – messbar, reproduzierbar und ohne manuellen Aufwand. Der Ablauf vom Einspielen der Testdaten bis zum Vergleich ist mit `demo/live_demo.sh` in einem Schritt reproduzierbar.

Zum eigenständigen Ausprobieren steht nach dem Start des Servers (siehe README im Repository) die interaktive Swagger-Oberfläche unter <http://localhost:8000/docs> im Browser zur Verfügung. Dort lassen sich alle Endpunkte – inklusive `GET /stats` und seiner Filterparameter – direkt über die Schaltfläche *Try it out* ausführen und die jeweilige Response sofort einsehen, ohne ein separates Tool wie `curl` zu benötigen.

5 Probleme, Ergebnis und Ausblick

5.1 Aufgetretene Probleme

Drei Probleme traten während der Implementierung auf. Erstens liefert die Aggregatfunktion für den Durchschnitt bei einer leeren Tabelle keinen numerischen Wert, sondern `None` – was bei der Weiterverarbeitung zu einem Laufzeitfehler führt. Das wurde durch eine explizite Prüfung vor der Typumwandlung behoben, sodass der Endpoint auch auf einer frischen Instanz ohne Daten fehlerfrei antwortet.

Zweitens verhält sich die Konvertierung des Zeitstempels auf den reinen Datumsteil in SQLite (Entwicklungsumgebung) und PostgreSQL (Produktion) unterschiedlich. Dieses Problem trat nicht nur theoretisch auf, sondern zeigte sich beim lokalen Testen direkt als Laufzeitfehler: Die ursprüngliche Umsetzung mit `cast(created_at, Date)` scheiterte in SQLite an der internen Verarbeitung des Datumswerts. Die Lösung war der Wechsel zu `func.date()`, einer Funktion die sowohl in SQLite als auch in PostgreSQL nativ unterstützt wird und in beiden Umgebungen zuverlässig einen String liefert – ohne separate Codepfade für die zwei Datenbanken.

Drittens wurde der Konfidenzfilter initial mit WHERE statt HAVING umgesetzt, was die Korrektheit der Ergebnisse beeinträchtigt hätte – der Fehler fiel beim manuellen Vergleich mit einer direkten Datenbankabfrage auf.

5.2 Ergebnis

Mit `GET /stats` wird die bestehende Datenbasis von PixelWise erstmals analytisch nutzbar. Der Endpoint aggregiert Predictions nach Ziffer, berechnet Durchschnittskonfidenz, unterstützt Filterung auf Gruppenebene und liefert eine tagesweise Zeitreihe. Durch den Datenbankindex auf `created_at` skaliert die Abfrage logarithmisch statt linear – was bei wachsendem Datenvolumen einen wesentlichen Unterschied macht. Der systemd-Timer sorgt dafür, dass täglich ein Snapshot gespeichert wird, und schafft damit die Voraussetzung für reproduzierbare Modellvergleiche – wie in Abbildung 1 gezeigt, hat sich dies in der praktischen Erprobung bestätigt.

5.3 Ausblick

Die naheliegendste Erweiterung wäre ein Feedback-Endpoint, über den Nutzer eine falsche Prediction korrigieren können – also die tatsächliche Ziffer mitteilen, wenn das Modell falsch lag. Diese korrekten Labels würden in einer separaten Tabelle gespeichert und könnten mit den Predictions verknüpft werden. Aus dieser Verknüpfung lässt sich eine Konfusionsmatrix berechnen: eine Gegenüberstellung von vorhergesagter und tatsächlicher Ziffer, die zeigt bei welchen Verwechslungen das Modell systematisch scheitert – etwa ob eine Vier häufig als Neun klassifiziert wird. Diese Erkenntnisse können gezielt in die Zusammenstellung von Trainingsdaten einfließen und so zur Verbesserung des Modells beitragen. Ein vollständiges Neutraining des Modells erfordert jedoch deutlich mehr als Feedback-Daten allein – insbesondere eine repräsentative und ausreichend große Datenbasis sowie die entsprechende Trainingsinfrastruktur.

Quellen

- Schutera, M. (2026). *Full Stack Handwerk – Unfinished Lecture Notes*. DHBW Ravensburg. <https://github.com/Quillstacks/LectureMaterial>
- PostgreSQL Global Development Group (2024). *PostgreSQL 16 – EXPLAIN*. <https://www.postgresql.org/docs/16/sql-explain.html>
- PostgreSQL Global Development Group (2024). *PostgreSQL 16 – Window Functions*. <https://www.postgresql.org/docs/16/tutorial-window.html>
- SQLAlchemy Documentation (2024). *Column Elements – func*. <https://docs.sqlalchemy.org/en/20/core/functions.html>
- systemd Documentation (2024). *systemd.timer*. <https://www.freedesktop.org/software/systemd/man/systemd.timer.html>